Daily Quant Digest: 2026-10-16

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Inequalities (Markov, Chebyshev, Cantelli, Jensen, Cauchy-Schwarz, Chernoff/Hoeffding); bounding 70+ heads in 100 flips, and volatility drag	Medium
Derivatives	Interest rate swaps: SOFR OIS mechanics, pricing, DV01, convexity and swap spreads	Medium
Algorithms & DS	Linked lists: reverse nodes in k-group, in place	Medium-Hard

---

1. Probability & Statistics: The Inequality Toolkit

Interviewers love inequalities because they test whether you understand *why* a tail is small, not just how to look it up. Every inequality below is one of two ideas: **bound an indicator by a nicer function** (Markov family) or **convexity** (Jensen family).

1.1 Markov: the mother inequality

Theorem. If $X \geq 0$ and $a > 0$, then $P(X \geq a) \leq \mathbb{E}[X]/a$.

Proof. Pointwise, $a\,\mathbf{1}\{X \geq a\} \leq X$ (if $X \geq a$ the left side is $a \leq X$; otherwise it is $0 \leq X$). Take expectations. $\square$

The proof is the whole lesson: we replaced the step function $\mathbf{1}\{x \geq a\}$ by the line x/a, which lies above it on $[0, \infty)$. Every other tail bound is the same move with a better upper function:

Upper function for $\mathbf{1}\{x \geq a\}$	Inequality	What it needs
x/a	Markov	$X \geq 0$, finite mean
$(x-\mu)^2/(a-\mu)^2$	Chebyshev	finite variance
$e^{t(x-a)}$, $t > 0$	Chernoff	finite MGF

The better the upper function hugs the indicator, the tighter the bound, and the more you have to know about X.

When is Markov tight? Equality needs $a\mathbf{1}\{X \geq a\} = X$ almost surely, i.e. $X \in \{0, a\}$. So for a two-point distribution on $\{0, a\}$ with $P(X = a) = \mu/a$, Markov is exact. This matters: no bound using only the mean can beat Markov.

1.2 Chebyshev and Cantelli

Apply Markov to $(X - \mu)^2$:

$$P(|X - \mu| \geq k) = P\big((X - \mu)^2 \geq k^2\big) \leq \frac{\sigma^2}{k^2}.$$

It is also tight among distributions with given mean and variance: put mass $\sigma^2/(2k^2)$ at $\mu \pm k$ and the rest at μ.

For **one-sided** tails, Chebyshev wastes a factor because it pays for both tails. **Cantelli's inequality** is the sharp one-sided version:

$$P(X - \mu \geq k) \leq \frac{\sigma^2}{\sigma^2 + k^2}.$$

Proof sketch. For any $u \geq 0$, $P(X - \mu \geq k) \leq P\big((X - \mu + u)^2 \geq (k + u)^2\big) \leq \frac{\sigma^2 + u^2}{(k+u)^2}$. Minimise over u: the optimum is $u = \sigma^2/k$, which gives $\sigma^2/(\sigma^2 + k^2)$. $\square$ (Shifting before squaring is a trick worth remembering.)

1.3 Chernoff: exponential Markov

For any $t > 0$, $x \mapsto e^{tx}$ is increasing and positive, so

$$P(X \geq a) = P(e^{tX} \geq e^{ta}) \leq e^{-ta}\, M_X(t) = \exp\big(-(ta - K_X(t))\big),$$

where $K_X = \log M_X$ is the cumulant generating function from yesterday's digest. The bound holds for **every** $t > 0$, so take the best one:

$$P(X \geq a) \leq \exp\big(-I(a)\big), \qquad I(a) = \sup_{t > 0} \big(ta - K_X(t)\big).$$

I is the **Legendre transform** (rate function) of the CGF.

Why it is so powerful for sums. If $S_n = X_1 + \cdots + X_n$ with i.i.d. X_i, independence gives $K_{S_n} = nK_X$, so

$$P(S_n \geq na) \leq e^{-n\, I(a)}.$$

The bound decays **exponentially in** n, while Chebyshev only gives $\sigma^2/(n(a-\mu)^2)$, i.e. $1/n$. Cramér's theorem says the exponent is exact: $\frac{1}{n}\log P(S_n \geq na) \to -I(a)$. Chernoff loses only a polynomial prefactor.

Bernoulli case (memorise this). For $X_i \sim \text{Bernoulli}(p)$ and $a > p$,

$$K(t) = \log(1-p+pe^t), \qquad e^{t^*} = \frac{a(1-p)}{p(1-a)}, \qquad I(a) = D(a\,\|\,p) = a\log\frac{a}{p} + (1-a)\log\frac{1-a}{1-p}$$

the **Kullback-Leibler divergence** between coins with bias a and p. The probability of a "wrong-looking" sample decays like $e^{-n\cdot\mathrm{KL}}$: this is the bridge between large deviations, hypothesis testing and information theory.

Hoeffding is the convenient, slightly weaker cousin. If $X_i \in [\ell_i, u_i]$ are independent (not necessarily identical), Hoeffding's lemma says $K_{X_i-\mu_i}(t) \leq t^2(u_i-\ell_i)^2/8$, and optimising t gives

$$P(S_n - \mathbb{E}S_n \geq k) \leq \exp\left(-\frac{2k^2}{\sum_i (u_i-\ell_i)^2}\right).$$

For fair coins this is $e^{-2k^2/n}$. It only uses boundedness, so it applies to anything bounded, which is why it is the workhorse in ML generalisation bounds and in backtests ("how many trades before a 55% hit rate is distinguishable from 50%?").

1.4 Jensen: convexity

Theorem. If φ is convex and X, $\varphi(X)$ are integrable, then $\varphi(\mathbb{E}X) \leq \mathbb{E}\,\varphi(X)$.

Proof. A convex function lies above each of its supporting lines. Take the line at $\mu = \mathbb{E}X$: there is a slope c with $\varphi(x) \geq \varphi(\mu) + c(x-\mu)$ for all x. Put $x = X$ and take expectations; the linear term vanishes. $\square$ Equality holds iff X is a.s. constant or φ is linear on the support of X.

Jensen is everywhere in finance:
- **Options have time value**: $\mathbb{E}[(S_T-K)^+] \geq (\mathbb{E}S_T - K)^+$ because $x \mapsto (x-K)^+$ is convex. Volatility can only add value to a convex payoff.
- **Volatility drag**: log is concave, so $\mathbb{E}[\log(1+R)] \leq \log(1+\mathbb{E}R)$. Expected growth is below the log of expected return, roughly $\mu - \sigma^2/2$. (The interview question below makes this concrete.)
- **Convexity adjustments**: $\mathbb{E}[1/(1+r)] \geq 1/(1+\mathbb{E}r)$, which is why futures rates and forward rates differ (curriculum item for later).
- **Siegel's paradox** in FX: $\mathbb{E}[1/X] \geq 1/\mathbb{E}[X]$, so the forward cannot be an unbiased predictor of the spot in both USDJPY and JPYUSD at once.
- **Power-mean / AM-GM**: $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$ gives geometric mean $\leq$ arithmetic mean.

1.5 Cauchy-Schwarz

Theorem. $\big(\mathbb{E}[XY]\big)^2 \le \mathbb{E}[X^2]\,\mathbb{E}[Y^2]$.

Proof. For every real λ, $0 \le \mathbb{E}[(X-\lambda Y)^2] = \mathbb{E}X^2 - 2\lambda\mathbb{E}XY + \lambda^2\mathbb{E}Y^2$. A non-negative quadratic in λ has discriminant ≤ 0. $\square$

Applied to $X - \mathbb{E}X$ and $Y - \mathbb{E}Y$ it says $|\operatorname{Cov}(X,Y)| \le \sigma_X\sigma_Y$, i.e. $|\rho| \le 1$, with equality iff Y is an affine function of X. It is also the reason a correlation matrix must be positive semidefinite: covariance is an inner product.

A nice companion is **Paley-Zygmund**, a *lower* bound from Cauchy-Schwarz: for $X \ge 0$ and $0 \le \theta \le 1$,

$$P(X > \theta\,\mathbb{E}X) \ge (1-\theta)^2\frac{(\mathbb{E}X)^2}{\mathbb{E}X^2}.$$

(Write $\mathbb{E}X \le \theta\mathbb{E}X + \mathbb{E}[X\mathbf{1}\{X > \theta\mathbb{E}X\}]$ and apply Cauchy-Schwarz to the last term.) Upper bounds say "rarely large"; this says "not always small".

1.6 Pitfalls

- **Markov needs $X \ge 0$.** Applied to a signed variable it is simply false.
- **Getting Jensen's direction wrong.** Draw the picture: for convex φ the chord lies above the curve, so the average of φ beats φ of the average.
- **Chernoff needs a finite MGF.** For heavy tails (Pareto, Student-t, many return series) $M_X(t) = \infty$ for every $t > 0$. Then you are back to polynomial bounds (Markov on $|X|^p$), and large deviations behave differently: one big jump, not many small ones, causes the tail event.
- **Chernoff and Hoeffding need independence** (or a martingale structure, as in Azuma). Correlated trades do not get e^{-n} concentration.
- **A bound is not an estimate.** Chebyshev can be off by orders of magnitude. Use bounds to rule things out and use the CLT or an exact calculation to estimate.
- **The CLT is bad in the far tail.** It is a statement about $O(\sqrt{n})$ deviations. For $O(n)$ deviations, Chernoff/Cramér gives the right exponent and the normal approximation can be badly wrong in relative terms.

1.7 Interview question

> **(a)** You flip a fair coin 100 times. Bound the probability of getting at least 70 heads using Markov, Chebyshev (or Cantelli), and a Chernoff/Hoeffding bound. Then give your best estimate of the true probability and compare.
>
> **(b)** A strategy each period either gains 50% or loses 40% with equal probability, independently. Its expected return per period is $+5\%$. Would you run it with all your capital, period after period? What does your wealth typically look like after 100

> periods?

1.8 Solution

Set-up. $X \sim \mathrm{Bin}(100, \tfrac{1}{2})$, so $\mu = 50$, $\sigma^2 = np(1-p) = 25$, $\sigma = 5$. We want $P(X \geq 70)$, a deviation of $20 = 4\sigma$.

Markov. $X \geq 0$, so $P(X \geq 70) \leq 50/70 \approx 0.714$. Correct but almost useless: Markov only knows the mean.

Chebyshev. $P(X \geq 70) \leq P(|X - 50| \geq 20) \leq 25/400 = 0.0625$.

Cantelli (one-sided). $P(X - 50 \geq 20) \leq 25/(25 + 400) = 1/17 \approx 0.0588$. A small gain over Chebyshev: the variance-only bounds cannot do much better because the extremal distributions are three-point, nothing like a binomial.

Hoeffding. $k = 20$, each flip in $[0,1]$: $P \leq e^{-2 \cdot 400/100} = e^{-8} \approx 3.35 \times 10^{-4}$.

Chernoff (optimal t). With $a = 0.7$, $p = 0.5$:

$$e^{t^*} = \frac{0.7 \times 0.5}{0.5 \times 0.3} = \frac{7}{3}, \quad t^* = \log \tfrac{7}{3} \approx 0.847.$$

The rate is

$$D(0.7 \,\|\, 0.5) = 0.7 \log 1.4 + 0.3 \log 0.6 = 0.7(0.33647) + 0.3(-0.51083) = 0.08228,$$

so $P \leq e^{-8.228} \approx 2.67 \times 10^{-4}$. Sanity check of the per-flip factor: $e^{-0.7t^*}(\tfrac{1}{2} + \tfrac{1}{2}e^{t^*}) = (3/7)^{0.7} \times \tfrac{5}{3} \approx 0.5526 \times 1.6667 \approx 0.9210 = e^{-0.08228}$. Consistent.

Note Chernoff beats Hoeffding (8.23 vs 8 in the exponent) because Hoeffding uses the worst-case variance for $[0,1]$ variables and a quadratic approximation to the KL divergence: $D(a\|p) \geq 2(a-p)^2 = 0.08$.

Best estimate. Normal approximation with continuity correction: $z = (69.5 - 50)/5 = 3.9$, so $P \approx \bar{\Phi}(3.9) \approx 4.8 \times 10^{-5}$. The exact binomial tail is

$$P(X \geq 70) = 2^{-100} \sum_{j=70}^{100} \binom{100}{j} \approx 3.93 \times 10^{-5}.$$

Method	Bound / estimate
Markov	0.714
Chebyshev	0.0625
Cantelli	0.0588
Hoeffding	3.35×10^{-4}
Chernoff (KL)	2.67×10^{-4}
Normal, continuity corrected	4.8×10^{-5}
Exact	$\mathbf{3.93 \times 10^{-5}}$

Why Chernoff is still off by about 7x. The exact asymptotic (Bahadur-Rao for a lattice variable) adds a polynomial prefactor:

$$P(S_n \geq na) \approx \frac{e^{-nD(a\|p)}}{\sqrt{2\pi n a(1-a)}} \cdot \frac{1}{1-e^{-t^*}}.$$

Here $\sqrt{2\pi \cdot 100 \cdot 0.21} \approx 11.49$ and $1/(1-3/7) = 1.75$, giving $2.67 \times 10^{-4} \times 1.75/11.49 \approx 4.07 \times 10^{-5}$, within 4% of the exact answer. That is a great "extra credit" remark: the exponent is right, the prefactor is $\Theta(1/\sqrt{n})$.

Interview trick. No calculator: a 4σ one-sided normal tail is about 3×10^{-5} (memorise $\bar{\Phi}(2) \approx 2.3\%$, $\bar{\Phi}(3) \approx 0.13\%$, $\bar{\Phi}(4) \approx 3.2 \times 10^{-5}$). Say "roughly 1 in 25,000, Chebyshev only guarantees 1 in 16, Chernoff guarantees about 1 in 3,700".

(b) Volatility drag. Arithmetic expectation per period: $\frac{1}{2}(1.5) + \frac{1}{2}(0.6) = 1.05$, so $\mathbb{E}[W_{100}] = 1.05^{100} \approx 131.5$ times the starting wealth. But growth compounds through logs. By Jensen with concave log:

$$\mathbb{E}[\log(1+R)] = \tfrac{1}{2}\log 1.5 + \tfrac{1}{2}\log 0.6 = \tfrac{1}{2}\log 0.9 \approx -0.0527 < 0 < \log 1.05.$$

By the LLN, $\frac{1}{n}\log W_n \to -0.0527$ almost surely, so wealth goes to zero. The **median** outcome after 100 periods is 50 up and 50 down: $1.5^{50} \times 0.6^{50} = 0.9^{50} \approx 0.0052$, i.e. you lose 99.5% of your capital. The mean of 131.5 is carried by a vanishingly small set of very lucky paths.

The fix is to bet a fraction f of capital: $g(f) = \frac{1}{2}\log(1+0.5f) + \frac{1}{2}\log(1-0.4f)$. Setting $g'(f) = 0$ gives $\frac{0.5}{1+0.5f} = \frac{0.4}{1-0.4f}$, so $f^* = 0.25$, with $g(0.25) \approx +0.0062 > 0$. That is the Kelly criterion, coming later in the curriculum.

Sanity checks. (i) Every bound is above the exact value and they are ordered by how much they know. (ii) $D(a\|p) \geq 2(a-p)^2$ (Pinsker) confirms Chernoff is at least as good as Hoeffding. (iii) For (b), the approximation $\mu - \sigma^2/2 = 0.05 - 0.45^2/2 \approx -0.051$ agrees with -0.0527.

---

2. Derivatives: Interest Rate Swaps

2.1 What a swap is

A plain vanilla interest rate swap (IRS) exchanges a stream of **fixed** coupons for a stream of **floating** coupons on the same notional. The notional is never exchanged. The **payer** pays fixed and receives floating; the **receiver** receives fixed and pays floating.

- **Receiver** = long duration, like owning a bond financed at the floating rate. Gains when rates fall.
- **Payer** = short duration. Gains when rates rise.

Since the LIBOR transition, the USD benchmark is the **SOFR OIS swap**: the floating leg pays daily SOFR compounded in arrears over each period. Standard terms for a USD SOFR swap: both legs pay **annually**, ACT/360, payment 2 business days after period end, spot start T+2. (In HKD, the analogous products are HIBOR swaps and HONIA OIS.)

2.2 Who trades them and why

Participant	Typical position	Reason
Corporate with floating-rate loan	Payer	Lock in a fixed borrowing cost
Corporate issuing fixed bonds	Receiver	Swap fixed debt to floating to match floating assets or a view
Pension funds, insurers	Receiver at the long end	Liabilities are long-dated; they need duration without buying bonds
Banks (ALM)	Both	Hedge fixed-rate mortgages and deposits
Mortgage servicers, MBS hedgers	Pay or receive dynamically	Hedge prepayment-driven negative convexity
Hedge funds, dealers	Both	Curve trades (2s10s), swap spread trades, macro views

2.3 Quoting, execution and settlement

- Quoted as the **par swap rate**: the fixed rate that makes the swap worth zero today, e.g. “5Y SOFR 3.650 / 3.655”.
- Executed electronically on SEFs (swap execution facilities) or voice, and most standard swaps must be **centrally cleared** at LCH SwapClear or CME. The CCP becomes counterparty to both sides; each side posts initial margin and settles variation margin daily.
- Risk is quoted in **DV01** (dollar value of a 1bp move), so a client will often say “I want to receive $50k DV01 of 5s” rather than naming a notional.

2.4 Pricing from first principles

Let $D(t)$ be the discount factor to time t, and fixed payment dates $t_1, \dots, t_n$ with accrual fractions τ_i.

Fixed leg. It is just a set of known cash flows:

$$PV_{\text{fix}} = K \cdot N \sum_{i=1}^{n} \tau_i D(t_i) = K \cdot N \cdot A, \qquad A = \sum_i \tau_i D(t_i),$$

where A is the **annuity** (or PV01 per unit rate).

Floating leg: the replication argument. Invest \$1 at t_{i-1} and roll it overnight at SOFR until t_i. You receive $\prod_j (1 + r_j \delta_j) = 1 + (\text{compounded SOFR coupon})$ at t_i. So "\$1 at t_{i-1}" is worth the same as "\$1 plus the floating coupon at t_i". Hence

$$PV(\text{coupon}_i) = D(t_{i-1}) - D(t_i).$$

Summing, the series **telescopes**:

$$PV_{\text{float}} = N \sum_{i=1}^{n} \big(D(t_{i-1}) - D(t_i) \big) = N \big(1 - D(t_n) \big)$$

for a spot-starting swap ($D(t_0) = 1$). You never need to forecast a single SOFR fixing. (With a separate discount curve and projection curve, as in the old LIBOR world, this neat telescoping breaks and you need forward rates from the projection curve.)

Par rate. Set $PV_{\text{fix}} = PV_{\text{float}}$:

$$\boxed{S = \frac{1 - D(t_n)}{A}}$$

This is the same formula as the par yield of a bond: a receiver swap is economically a par bond minus a floating rate note.

Value of an existing swap with fixed rate K when today's par rate for the remaining maturity is S:

$$V_{\text{receiver}} = N \cdot (K - S) \cdot A.$$

You are long the difference between your locked rate and the market rate, on an annuity.

DV01. Bumping the par rate by 1bp changes value by approximately $N \cdot A \cdot 0.0001$. A useful mental rule: DV01 per \$1M notional $\approx$ \$100 $\times$ (annuity in years), so roughly \$450 per \$1M for a 5Y and \$820 per \$1M for a 10Y at today's rate levels.

Convexity. A receiver swap is long convexity (like a bond): as rates fall, A increases and the DV01 grows, so gains on a rally exceed losses on an equal selloff. A payer is short convexity.

2.5 Worked scenario: a 5Y SOFR receiver

Assume (illustrative) annually compounded zero rates and simplify accruals to $\tau_i = 1$:

t (years)	Zero rate	$D(t) = (1+z)^{-t}$
1	3.90%	0.962464
2	3.70%	0.929913
3	3.60%	0.899333
4	3.60%	0.868082
5	3.65%	0.835898

Step 1: annuity. $A = 0.962464 + 0.929913 + 0.899333 + 0.868082 + 0.835898 = 4.495691$.

Step 2: par rate. $S = (1 - 0.835898)/4.495691 = 0.164102/4.495691 = 3.6502\%$. Notice it is close to the 5Y zero rate because the curve is fairly flat.

Step 3: trade. You receive fixed at 3.6502% on \$100M for 5 years. Value at inception: 0.

Step 4: DV01. $100{,}000{,}000 \times 4.495691 \times 0.0001 \approx \$44{,}957$ per bp. In words: you are long \$45k per bp, the same rate risk as owning roughly \$100M of a 5Y par bond.

Step 5: market moves. The whole zero curve shifts up 10bp. Revalue:
- new par rate 3.7502% (it moved 10.0bp),
- new value $V = N\big(K \cdot A' - (1 - D'(5))\big) = -\$448{,}351$.

Compare with the DV01 estimate $-10 \times 44{,}957 = -\$449{,}569$. The small gap is convexity: as rates rose the annuity shrank, so the loss is a bit smaller than linear. If instead the curve shifts **down** 10bp the value is $+\$450{,}891$. The asymmetry, $450{,}891 - 448{,}351 = \$2{,}540$, is your convexity gain on a $\pm$10bp straddle of moves.

Step 6: an off-market swap. Suppose instead you had an old swap receiving 4.25% with exactly 5 years left on \$100M. Its value today is

$$V = 100\text{M} \times (4.25\% - 3.6502\%) \times 4.495691 \approx +\$2.70\text{M}.$$

If you wanted to unwind, the dealer would pay you about this amount (less bid/offer). Equivalently, you could enter an offsetting payer at 3.6502% and lock in a 59.98bp annuity.

Step 7: swap spreads. Suppose the 5Y Treasury yields 3.85%. The **swap spread** is

$$\text{swap rate} - \text{Treasury yield} = 3.650\% - 3.850\% \approx -20\text{bp}.$$

Before 2008 USD swap spreads were positive (swaps referenced LIBOR, which carried bank credit risk). Since around 2015 they have been negative at the long end, and SOFR swaps embed essentially no credit premium. Drivers of negative spreads:
- **Balance sheet cost.** Holding a Treasury consumes dealer balance sheet (leverage ratio / SLR); a cleared swap consumes far less. Investors demand a higher yield on the Treasury to compensate.
- **Treasury supply.** Large deficits mean heavy issuance that needs to be absorbed.
- **Duration demand in swaps.** Pension funds and insurers receive fixed in long swaps, pushing swap rates down.

A **"long swap spread"** trade (betting spreads widen, i.e. become less negative) is: buy the Treasury, financed in repo, and pay fixed on a DV01-matched swap. Carry is roughly (Treasury yield $-$ swap rate) $-$ (repo rate $-$ SOFR) $= 20\text{bp} - 5\text{bp} = 15\text{bp}$ a year, or about \$150k per \$100M, if repo trades 5bp over SOFR. The catch is that it ties up balance sheet and can lose money for a long time if spreads keep tightening, which is exactly the regulatory story that keeps them negative. (This is a close cousin of yesterday's cash-futures basis trade.)

2.6 What can go wrong

- **Curve risk.** A 5Y receiver hedged with a 10Y payer is DV01 neutral but exposed to 5s10s steepening. Real books report risk by bucket ("key rate DV01s").
- **Fixing and basis risk.** A hedge on SOFR against an asset on Term SOFR, or USD against HKD HIBOR, leaves basis exposure.
- **Liquidity and margin.** Mark-to-market losses are paid daily in cash (variation margin). In 2022 UK pension funds (LDI) running leveraged receivers faced huge margin calls when gilt yields spiked and had to sell gilts, which pushed yields higher still.
- **Counterparty risk** on uncleared swaps, priced via CVA. Clearing replaces it with CCP and margin-liquidity risk.
- **Convexity for payers.** Mortgage hedgers who pay fixed get shorter duration in a selloff and must pay more, amplifying moves.

2.7 Likely interview questions

1. **"Why is the floating leg of an OIS swap worth $N(1 - D(T))$?"** Rolling \$1 overnight from today to T produces exactly the floating coupons plus \$1 at T. So the coupons are worth $1 - D(T)$ per unit notional; the sum telescopes and no forecasting is needed.

2. **“Roughly, what is the DV01 of $100M 10Y swap?”** Annuity about 8.2 years at ~3.8% rates, so about $82k per bp. (Quick rule: about $100 per $1M per year of annuity; annuity is a bit less than maturity because of discounting.)
3. **“How can swap spreads be negative if a swap has counterparty risk and a Treasury doesn’t?”** SOFR is a near risk-free rate and clearing removes most counterparty risk, while holding Treasuries consumes scarce dealer balance sheet and Treasury supply is large. Arbitraging the gap (buy Treasury, pay fixed) requires repo financing and balance sheet, so it is not free money, and the spread can stay negative.

---

3. Algorithms & Data Structures: Reverse Nodes in k-Group

3.1 Problem

Given the head of a singly linked list and an integer k, reverse the nodes of the list k at a time and return the new head. If the number of nodes remaining at the end is less than k, leave them in their original order. You may only change `next` pointers, not node values.

Constraints. $1 \le n \le 5000$, $1 \le k \le n$, $0 \le$ `val` ≤ 1000. Follow-up: use $O(1)$ extra memory.

Examples.
- `[1,2,3,4,5]`, $k = 2 \to$ `[2,1,4,3,5]`
- `[1,2,3,4,5]`, $k = 3 \to$ `[3,2,1,4,5]`
- `[1,2,3,4,5,6]`, $k = 3 \to$ `[3,2,1,6,5,4]`
- `[1,2,3,4,5]`, $k = 1 \to$ unchanged

This is a favourite at HFT firms because it is pure pointer discipline: no clever algorithm, but many ways to lose a node or create a cycle.

3.2 Brute force

Walk the list and push all node pointers into a `vector<ListNode*>`. Reverse each full block of k entries with `std::reverse`, then relink: `v[i]->next = v[i+1]`, and the last one to `nullptr`. That is $O(n)$ time but $O(n)$ extra memory, and it violates the follow-up. An alternative is a stack of size k: push k nodes, pop them to relink, which is $O(k)$ memory. Both are good to mention first, because they make the target clear.

3.3 The insight: one in-place reversal, done n/k times, plus careful stitching

You already know how to reverse a whole list in place with three pointers (`prev`, `cur`, `nxt`). Reversing a **segment** is the same loop with two changes:

1. **Stop at the segment boundary.** Remember `groupNext`, the node after the k-th node, and loop while `cur != groupNext`.

2. **Seed `prev` with `groupNext`** instead of `nullptr`. Then the first node of the group, which becomes its last, automatically points to the rest of the list.

What remains is stitching. Keep `groupPrev`, the node just before the group. After reversal:
- `groupPrev->next` must become the old k-th node (the new group head).
- The old first node (`groupStart`) is now the group's tail, so it becomes `groupPrev` for the next round.

A **dummy (sentinel) node** before the head makes the very first group identical to every other one: there is always a `groupPrev`.

Before reversing, check that k nodes actually exist by walking k steps from `groupPrev`; if you fall off the end, stop and leave the tail untouched.

Picture for `1 2 3 4 5`, $k = 3$:

```
dummy -> 1 -> 2 -> 3 -> 4 -> 5        groupPrev=dummy, kth=3, groupNext=4
reverse with prev seeded at 4:
          3 -> 2 -> 1 -> 4 -> 5
dummy -> 3 (stitch)                     groupPrev = 1 (old start)
from 1, walk 3 steps: 4, 5, null -> stop.  Result: 3 2 1 4 5
```

Complexity. Each node is visited twice (once by the "find k-th" walk, once by the reversal), so $O(n)$ time. Only a handful of pointers: $O(1)$ extra space. A recursive solution (reverse the first k, recurse on the rest) is shorter but uses $O(n/k)$ stack frames.

3.4 C++ solution

```cpp
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <vector>

using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    explicit ListNode(int v, ListNode* n = nullptr) : val(v), next(n) {}
};

// Reverses the nodes of the list k at a time and returns the new head.
// A final group with fewer than k nodes is left in its original order.
// Only next pointers are rewired: O(n) time, O(1) extra space.
ListNode* reverseKGroup(ListNode* head, int k) {
    if (head == nullptr || k <= 1) return head;

    ListNode dummy(0, head);    // sentinel on the stack: no special case for the head
    ListNode* groupPrev = &dummy;  // node just before the current group

    while (true) {
        // 1. Find the k-th node of this group (or discover there are fewer than k left).
        ListNode* kth = groupPrev;
        for (int i = 0; i < k && kth != nullptr; ++i) kth = kth->next;
        if (kth == nullptr) break;   // short tail: leave it alone
```

```
        ListNode* groupStart = groupPrev->next;  // becomes the group's tail after reversal
        ListNode* groupNext = kth->next;         // first node after the group

        // 2. Reverse the k nodes in place. Seeding prev with groupNext means the
        //    old first node ends up pointing to the rest of the list automatically.
        ListNode* prev = groupNext;
        ListNode* cur = groupStart;
        while (cur != groupNext) {
            ListNode* nxt = cur->next;
            cur->next = prev;
            prev = cur;
            cur = nxt;
        }

        // 3. Stitch: the node before the group now points at the old k-th node.
        groupPrev->next = kth;
        groupPrev = groupStart;
    }
    return dummy.next;
}
```

The full file (`2026-10-16.cpp`) adds `build`, `toVector` and `destroy` helpers, the example cases, and a randomised test of 2000 lists against a reference that applies `std::reverse` to blocks of a `vector`. It compiles cleanly with `-Wall -Wextra` and passes under AddressSanitizer (no leaks, no use-after-free).

3.5 Walkthrough of the C++

- **`struct ListNode` with a constructor.** In C++ a `struct` is a class whose members are public by default. The constructor uses a **member initialiser list** (`: val(v), next(n)`), which initialises members directly instead of assigning them in the body. `ListNode* n = nullptr` is a **default argument**, so `ListNode(5)` and `ListNode(5, p)` both work. `explicit` stops accidental implicit conversions such as `ListNode x = 5;`.
- **`nullptr`, not `NULL` or `0`.** `nullptr` has its own type (`std::nullptr_t`), so it cannot be confused with an integer in overload resolution.
- **The sentinel lives on the stack.** `ListNode dummy(0, head);` is a local object, not `new`-allocated, so it is destroyed automatically when the function returns. `&dummy` takes its address so it can be treated like any other node pointer. We return `dummy.next` (member access with `.` on an object) rather than `dummy->next` (which is for pointers). Returning `&dummy` itself would be a dangling pointer bug.
- **Pointer re-seating vs references.** All the work is reassigning `ListNode*` variables. A C++ reference (`ListNode&`) cannot be re-seated to refer to a different object after initialisation, which is why linked-list code uses pointers.
- **The order of the three-pointer loop matters.** Save `nxt = cur->next` *before* overwriting `cur->next`, otherwise the rest of the list is lost. A good habit is to read each line as "save, rewire, advance, advance".
- **`for (int i = 0; i < k && kth != nullptr; ++i)`.** The `&&` short-circuits, so we never dereference a null pointer.
- **`const ListNode*` in `toVector`.** A pointer to const: the function can walk the list but cannot modify nodes. The pointer itself can still move (`p = p->next`). `ListNode* const p` would be the opposite (fixed pointer, mutable node).

- **Manual memory: `new` and `delete`.** `build` allocates with `new`; `destroy` saves `next` before `delete`-ing each node (deleting first and then reading `head->next` is use-after-free). In production you might prefer `std::unique_ptr<ListNode>` for ownership, but note a naive `unique_ptr` chain destroys recursively and can overflow the stack on a very long list, so interviewers usually expect raw pointers here.
- **Aggregate initialisation of test cases.** `struct Case { vector<int> in; int k; vector<int> expected; };` has no constructor, so `{{1,2,3,4,5}, 2, {2,1,4,3,5}}` fills the members in order. `for (const auto& c : cases)` iterates by const reference, avoiding copying the vectors.
- **`std::reverse(v.begin() + i, v.begin() + i + k)`.** Iterators into a `vector` are random-access, so `+ i` jumps directly. The range is half-open, `[first, last)`, the standard STL convention.
- **`mt19937 rng(42)`.** The Mersenne Twister engine with a fixed seed gives reproducible random tests. `rng() % n` has a tiny modulo bias, fine for testing; use `uniform_int_distribution` when it matters.
- **`assert`.** Aborts with the failing expression if false. It is compiled out when `NDEBUG` is defined, so never put side effects inside it.

3.6 Variations to be ready for

- **Swap nodes in pairs**: the $k = 2$ special case.
- **Reverse a sublist between positions m and n**: one segment reversal with the same stitching.
- **Reverse the trailing short group too**: drop the "fewer than k" check.
- **Reverse groups starting from the end**: compute the length n first and skip n mod k nodes, then apply the same routine.
- **Detect a cycle and find its start (Floyd)**: slow and fast pointers meet inside the cycle; resetting one to the head and moving both one step at a time makes them meet at the cycle entrance, because the distance from head to entrance is congruent to the distance from the meeting point to the entrance modulo the cycle length.